

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Hoàng Hữu Đức	23020046
Nguyễn Nho Dương	23020034
Lê Minh Tuấn	23020149
Phạm Minh Thông	23020164

**CÔNG CỤ KIỂM THỬ
CHECKSTYLE**

BÁO CÁO BÀI TẬP LỚN

Bộ môn: Kiểm thử và đảm bảo chất lượng phần mềm

HÀ NỘI - 2025

LỜI CAM ĐOAN

Chúng em xin cam đoan: Báo cáo này là kết quả của công việc nghiên cứu và thực hiện của nhóm. Những gì chúng em viết ra hoàn toàn trung thực, chính xác và không có sự sao chép từ các tài liệu, không sử dụng kết quả của người khác mà không trích dẫn cụ thể. Các dữ liệu, kết quả thực nghiệm, và phân tích được trình bày trong báo cáo đều là chính xác và thực tế, không được làm giả hoặc bao gồm bất kỳ thông tin sai lệch nào. Các tài liệu tham khảo được liệt kê đầy đủ và chính xác trong phần “tài liệu tham khảo” của báo cáo. Nếu vi phạm những điều trên, chúng em xin hoàn toàn chịu trách nhiệm về những hậu quả pháp lý liên quan theo quy định của Trường Đại học Công nghệ - ĐHQGHN.

Hà Nội, ngày 8 tháng 12 năm 2025

Sinh viên

Hoàng Hữu Đức

MỤC LỤC

Lời cam đoan	i
Mục lục	ii
Danh mục hình ảnh	iv
Danh mục bảng	v
Chương 1. Giới thiệu	1
1.1. Bối cảnh và lý do chọn đề tài	1
1.1.1. Bối cảnh:	1
1.1.2. Lý do chọn đề tài:	1
1.2. Mục tiêu nghiên cứu	2
1.3. Phương pháp nghiên cứu	2
1.4. Kiến thức liên quan	3
1.5. Cấu trúc của báo cáo	3
Chương 2. Tổng quan về Checkstyle	4
2.1. Giới thiệu chung	4
2.2. Kiến trúc	5
2.2.1. Tổng quan	5
2.2.2. Cây cấu hình	5
2.2.3. Cây cú pháp	6
2.3. Luồng hoạt động	6
2.3.1. Tổng quan	6
2.3.2. TreeWalker	8
2.3.3. Các loại kiểm tra khác	8
Chương 3. Áp dụng Checkstyle phân tích dự án thực tế	9

3.1. Lựa chọn dự án và thiết lập môi trường phân tích	9
3.1.1. Lựa chọn dự án	9
3.1.2. Thiết lập môi trường	9
3.2. Cấu hình và thực thi kiểm thử	10
3.2.1. Cấu hình quy tắc kiểm thử	10
3.2.2. Thực thi kiểm thử	12
3.3. Kết quả kiểm thử	12
3.4. Phân tích kết quả	14
3.5. Đề xuất khắc phục	15
Chương 4. Kết luận	18
4.1. Đánh giá hiệu quả	18
4.2. Kết luận	18

DANH MỤC HÌNH ẢNH

Hình 3.1 Tổng quan các lỗi vi phạm trong dự án MegaBasterd	14
--	----

DANH MỤC BẢNG

Bảng 3.1 So sánh giữa Google Checks và Sun Checks	10
---	----

Chương 1

Giới thiệu

1.1. Bối cảnh và lý do chọn đề tài

Trong thời đại phát triển nhanh chóng của công nghệ thông tin, chất lượng phần mềm là một yếu tố quyết định đến sự thành công của các dự án phát triển. Tuy nhiên, việc duy trì tiêu chuẩn mã hóa và đảm bảo mã nguồn tuân thủ các quy tắc thiết lập là một thách thức lớn, đặc biệt khi các đội phát triển có nhiều thành viên với các thói quen lập trình khác nhau.

1.1.1. Bối cảnh:

Ngôn ngữ lập trình Java là một trong những ngôn ngữ phổ biến nhất hiện nay, được sử dụng rộng rãi trong phát triển các ứng dụng doanh nghiệp, hệ thống Backend, và các dự án mã nguồn mở quy mô lớn. Tuy nhiên, Java cũng là ngôn ngữ có tính linh hoạt cao, cho phép các lập trình viên viết code theo nhiều cách khác nhau. Điều này dẫn đến:

- Mã nguồn thiếu tính nhất quán trong định dạng và cấu trúc.
- Khó khăn trong việc code review và bảo trì.
- Rủi ro cao trong việc phát sinh các lỗi tiềm ẩn liên quan đến định dạng và quy ước đặt tên.
- Tăng chi phí bảo trì và cập nhật phần mềm.

Để giải quyết những vấn đề trên, các công cụ phân tích mã tĩnh (static code analysis tools) như Checkstyle, SpotBugs, PMD, v.v. đã được phát triển. Trong đó, **Checkstyle** là một công cụ chuyên biệt dành riêng cho ngôn ngữ Java, giúp lập trình viên tự động hóa quá trình kiểm tra và thực thi các tiêu chuẩn mã hóa.

1.1.2. Lý do chọn đề tài:

Nhóm chọn Checkstyle làm chủ đề của báo cáo vì những lý do sau:

1. Checkstyle là công cụ thiết yếu trong quy trình kiểm thử phần mềm, giúp phát hiện các vi phạm về tiêu chuẩn mã hóa từ sớm, trước khi code được merge vào nhánh chính.
2. Công cụ này được sử dụng rộng rãi trong các dự án mã nguồn mở lớn như Apache, Eclipse, Maven, v.v. Việc hiểu rõ cách sử dụng Checkstyle sẽ giúp ích rất nhiều cho các lập trình viên trong quá trình phát triển chuyên nghiệp.
3. Checkstyle không chỉ giúp duy trì tính nhất quán của code, mà còn góp phần nâng cao chất lượng tổng thể của phần mềm, giảm thiểu lỗi tiềm ẩn, và cải thiện khả năng bảo trì.
4. Checkstyle có thể được tích hợp dễ dàng vào các hệ thống CI/CD như Jenkins, GitLab CI, GitHub Actions, v.v. để tự động kiểm tra mã nguồn trong mỗi lần commit hoặc pull request.
5. Checkstyle cho phép người dùng tùy chỉnh các quy tắc kiểm thử thông qua file cấu hình XML, phù hợp với quy chuẩn khác nhau của từng tổ chức hay dự án.

1.2. Mục tiêu nghiên cứu

Trong báo cáo này, nhóm sẽ tìm hiểu về công cụ Checkstyle và cách ứng dụng vào thực tiễn.

1. Tìm hiểu chi tiết về công cụ Checkstyle, bao gồm kiến trúc, nguyên tắc hoạt động, và các tính năng chính.
2. Học cách cấu hình và sử dụng Checkstyle để kiểm tra mã nguồn Java.
3. Áp dụng Checkstyle vào phân tích một dự án Java thực tế và đưa ra các đề xuất khắc phục.
4. Rút ra những bài học và kinh nghiệm hữu ích về việc đảm bảo chất lượng phần mềm thông qua static code analysis.

1.3. Phương pháp nghiên cứu

Để thực hiện đề tài này, nhóm đã áp dụng các phương pháp nghiên cứu sau:

- Nghiên cứu tài liệu: Tìm hiểu từ các tài liệu chính thức của Checkstyle, các bài viết công bố khoa học, và các blog về công cụ này.

- Phân tích thực nghiệm: Cài đặt và sử dụng Checkstyle trên một dự án Java thực tế (MegaBasterd) để kiểm tra mã nguồn và phân tích kết quả.
- So sánh và đánh giá: So sánh các bộ quy tắc khác nhau (Google Checks, Sun Checks), so sánh giữa Checkstyle và các công cụ phân tích mã tĩnh khác như PMD, SpotBugs để đưa ra nhận xét và đánh giá.

1.4. Kiến thức liên quan

- Static Code Analysis (Phân tích mã tĩnh): Một kỹ thuật kiểm tra mã nguồn mà không cần chạy chương trình.
- Code Convention (Quy ước mã hóa): Các quy tắc thống nhất về cách trình bày code, định dạng, đặt tên, v.v.
- CI/CD (Continuous Integration/Continuous Deployment): Quy trình tự động hóa phát triển, kiểm thử, và triển khai phần mềm.
- AST (Abstract Syntax Tree): Cấu trúc dữ liệu biểu diễn cấu trúc cú pháp của mã nguồn dưới dạng cây.

1.5. Cấu trúc của báo cáo

Phần còn lại của báo cáo này được trình bày như sau:

- [Chương 2](#): Tổng quan về Checkstyle
- [Chương 3](#): Áp dụng Checkstyle phân tích dự án thực tế
- [Chương 4](#): Kết luận

Chương 2

Tổng quan về Checkstyle

2.1. Giới thiệu chung

Checkstyle là một công cụ phân tích mã tĩnh mã nguồn mở, được phát triển và thiết kế chuyên biệt cho ngôn ngữ Java. Công cụ này giúp lập trình viên viết mã Java tuân thủ tiêu chuẩn mã hóa trong phát triển phần mềm.

Checkstyle có thể hoạt động trên nhiều nền tảng như Windows, macOS, Linux/Unix. Ngôn ngữ phát triển chính: Java.

Mục đích sử dụng:

- **Thực thi tiêu chuẩn code:** Giúp lập trình viên viết mã Java tuân thủ theo các bộ quy tắc, ví dụ như Google Java Style hoặc Sun Code Conventions.
- **Tự động hóa kiểm tra style:** Giảm bớt công việc kiểm tra thủ công, tránh lỗi do con người.
- **Cải thiện chất lượng code:** Tăng tính nhất quán, dễ đọc, dễ bảo trì và tái sử dụng.
- **Phát hiện sớm lỗi:** Báo cáo vi phạm về định dạng, cấu trúc và thiết kế ngay trong quá trình phát triển hoặc build.

Tính năng hỗ trợ:

- **Tích hợp CI/CD:** Tạo báo cáo vi phạm chi tiết, tích hợp cùng Jenkins, Maven, Gradle,...
- **Tiêu chuẩn & quy tắc mã hóa:** Kiểm tra dựa trên quy ước được cấu hình trước; hỗ trợ quy tắc tùy chỉnh.
- **Hỗ trợ plugin:** Tích hợp với Eclipse, IntelliJ IDEA, NetBeans; kiểm tra ngay khi lập trình viên viết code.

2.2. Kiến trúc

2.2.1. Tổng quan

2.2.2. Cây cấu hình

Một cấu hình (Configuration) của Checkstyle quy định những module nào sẽ được sử dụng và áp dụng lên các tệp mã nguồn Java. Các module này được tổ chức theo dạng cây, trong đó gốc của cây luôn là module Checker. Dưới Checker là các nhóm module chính:

- File Set Checks: Các module sử dụng để kiểm tra các vi phạm từ các file đầu vào.
- Filters: các module dùng để lọc các sự kiện kiểm tra (audit events), từ đó quyết định các yêu cầu có được chấp nhận hay bị loại bỏ.
- Audit Listeners: các module nhận và báo cáo các sự kiện sau khi đã được kiểm duyệt.

Khi sử dụng Checkstyle để tiến hành kiểm thử, người dùng cần chỉ định một tập tin XML, trong đó các phần tử thể hiện cấu trúc cây module của cấu hình, các module được bật, và các thuộc tính được thiết lập cho từng module.

Một ví dụ:

```
<module name="Checker">
  <module name="JavadocPackage"/>
  <module name="TreeWalker">
    <property name="tabWidth" value="4"/>
    <module name="AvoidStarImport"/>
    <module name="ConstantName"/>
    ...
  </module>
</module>
```

Một module trong file cấu hình XML được biểu diễn bằng thẻ `<module>` cùng trường `name`. Ở ví dụ trên, ...

Với mỗi module, Checkstyle sẽ load các class được chỉ định trong trường `name` của các thẻ `<module>`, theo các nguyên tắc sau:

- Load trực tiếp một class theo tên đầy đủ của nó (ví dụ `com.puppycrawl.tools.checkstyle.TreeWalker`), sử dụng khi ta cần tích hợp các module bên thứ ba.
- Load một class được Checkstyle quy định sẵn (ví dụ, khi ta muốn load class `com.puppycrawl.tools.checkstyle.checks.AvoidStarImport`, ta chỉ cần chỉ định trường `name` là `AvoidStarImport`, khi đó công cụ sẽ tự động tìm class theo các package `com.puppycrawl.tools.checkstyle`, `com.puppycrawl.tools.checkstyle.filters`, `com.puppycrawl.tools.checkstyle.checks` và các subpackage khác của các package này trong Checkstyle distribution)
- Áp dụng các quy tắc trên cho tên module sau khi thêm hậu tố “Check” (ví dụ tải lớp `com.puppycrawl.tools.checkstyle.checks.ConstantNameCheck` cho phần tử `<module name="ConstantName"/>`)

Để điều chỉnh các hành vi của một module, ta có thể dùng thẻ `<properties>` với các trường `name` và `value`. Ví dụ, trong đoạn code trên, ta quy định độ rộng của một kí tự tab là 4 cho module `TreeWalker` cùng tất cả các module con của nó.

2.2.3. Cây cú pháp

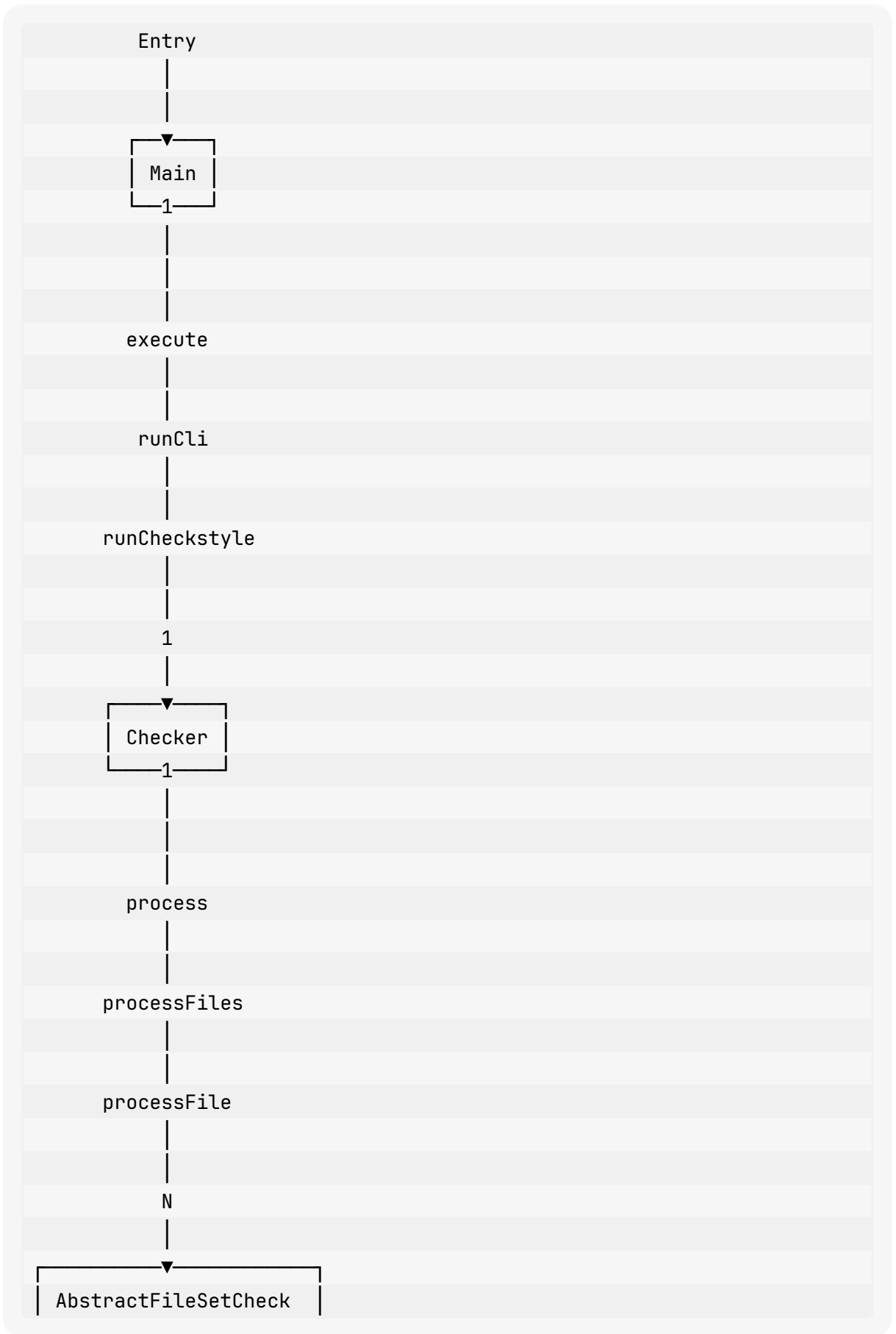
Để kiểm tra mã nguồn, hầu hết các tiêu chí lựa chọn sử dụng cây cú pháp (Abstract Syntax Tree) được dựng trực tiếp từ mã nguồn Java gốc. Một nút trên cây cú pháp được biểu diễn bằng lớp `DetailAST`, với cấu trúc như sau:

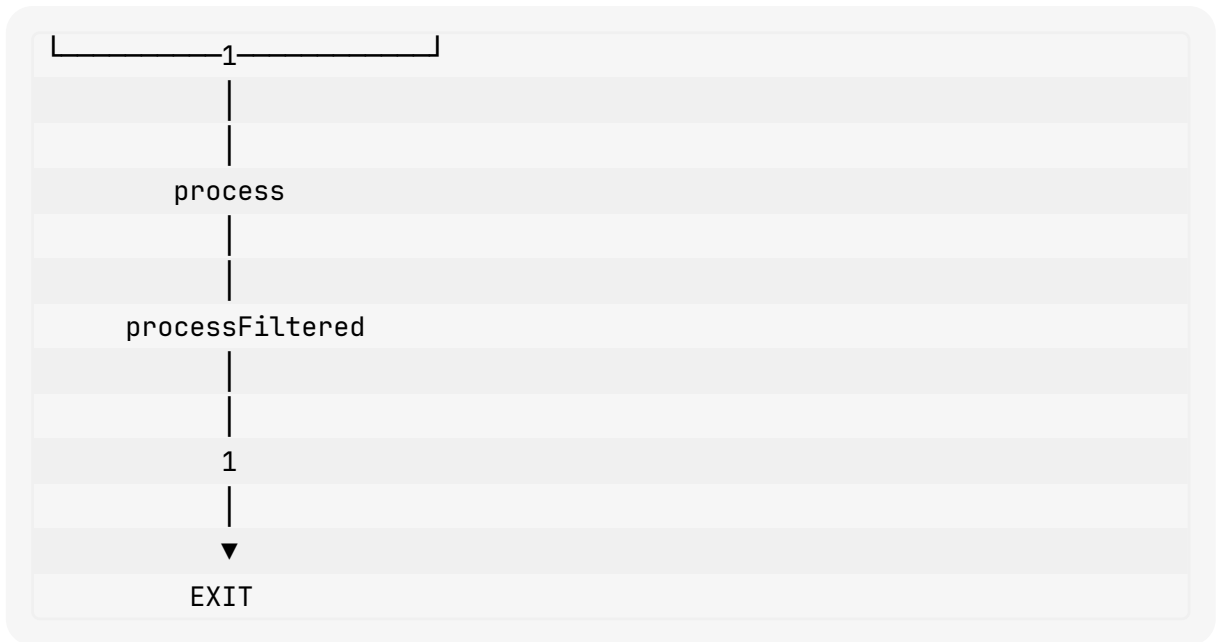
- `getType()`: Loại của nút, ví dụ `CLASS_DEF` thể hiện định nghĩa của một class, hay `METHOD_DEF` thể hiện định nghĩa của một phương thức. Chi tiết về các loại được thể hiện trong lớp `TokenTypes`
- `getText()`: Tên hoặc ký hiệu liên quan tới node.
- `getLineNo()` / `getColumnNo()`: Trả ra số dòng và vị trí trong dòng (đánh số từ 0), phục vụ cho việc tìm lỗi.
- Các thông tin liên quan đến cha/con trực tiếp của đỉnh hiện tại.

2.3. Luồng hoạt động

2.3.1. Tổng quan

Vẽ lại hình:





2.3.2. TreeWalker

Đầu tiên, lớp này tiến hành chuyển mã nguồn Java sang dạng cây sử dụng lớp `JavaParser`, trong đó sử dụng mã nguồn của ANTLR, một công cụ mã nguồn mở dùng để phân tích cú pháp.

Sau khi có được cấu trúc cây, tiến hành duyệt cây theo phương pháp DFS khử đệ quy. Khi bắt đầu thăm một nút, ta gọi hàm `visitToken()` của tất cả các check được định nghĩa một cách tuần tự để kiểm tra các điều kiện. Bên cạnh những điều kiện có thể kiểm tra trực tiếp, còn có những điều kiện cần phải biết hết thông tin của cây con để kiểm tra kết quả (ví dụ như độ dài phương thức, hay độ phủ của các biến), nên sau khi thăm toàn bộ cây con gốc u , ta cần phải gọi hàm `leaveToken()` của tất cả các check để đảm bảo tất cả điều kiện đã được kiểm tra.

Cuối cùng, các vi phạm được tổng hợp và gửi lại cho Checker, thành phần chịu trách nhiệm thu thập và xuất kết quả đầu ra.

2.3.3. Các loại kiểm tra khác

Ngoài ra, một số loại kiểm tra có thể không cần dùng đến cây cấu trúc mà có thể kiểm tra trực tiếp trên mã nguồn gốc (ví dụ như kiểm tra độ dài của phương thức, hay của file...). Các lớp thể hiện các tiêu chí kiểm tra như vậy được kế thừa từ lớp `AbstractFileSetCheck` (trừ lớp `TreeWalker`).

Chương 3

Áp dụng Checkstyle phân tích dự án thực tế

3.1. Lựa chọn dự án và thiết lập môi trường phân tích

3.1.1. Lựa chọn dự án

MegaBasterd là một trình quản lý tải xuống mã nguồn mở được viết bằng ngôn ngữ Java . Nó được thiết kế để đơn giản hóa quá trình tải xuống các tệp lớn từ dịch vụ lưu trữ đám mây Mega.nz.

Các tính năng chính của MegaBasterd bao gồm:

- Hỗ trợ tải xuống từ Mega.nz với tốc độ cao.
- Hỗ trợ tải xuống hàng loạt (batch download).
- Cung cấp một giao diện dễ sử dụng để quản lý các lần tải xuống.

Dự án có sẵn trên Github tại: <https://github.com/tonikelope/megabasterd/>

Lí do lựa chọn dự án MegaBasterd:

- Dự án mã nguồn mở, được viết bằng Java, phù hợp để phân tích bằng Checkstyle.
- Quy mô vừa phải, có đủ số lượng file và dòng code để phân tích.
- Được đóng góp bởi nhiều contributor với thói quen code khác nhau, giúp kiểm thử hiệu quả hơn.

3.1.2. Thiết lập môi trường

Có 3 cách để tích hợp Checkstyle vào quy trình phát triển phần mềm, dự án:

- Sử dụng Checkstyle thông qua command line.
- Tích hợp Checkstyle vào IDE (Eclipse, IntelliJ IDEA).
- Tích hợp Checkstyle vào hệ thống build tự động (Maven, Gradle).

Trong khuôn khổ báo cáo này, nhóm sẽ sử dụng cách thứ nhất – Chạy Checkstyle thông qua command line để thực hiện phân tích mã nguồn dự án MegaBasterd.

Công cụ và phiên bản được sử dụng:

- JDK: 25.0.1
- Checkstyle: 12.1.1
- IDE: IntelliJ IDEA Ultimate 2024.3

Các bước cài đặt Checkstyle:

1. Truy cập trang [Github Release của Checkstyle](#).
2. Tải về file JAR phiên bản mới nhất (tại thời điểm viết báo cáo là 12.1.1).
3. Đặt file JAR vào một thư mục cố định trên máy tính, ví dụ:
`C:\checkstyle\checkstyle-12.1.1-all.jar`.

3.2. Cấu hình và thực thi kiểm thử

3.2.1. Cấu hình quy tắc kiểm thử

Trên trang Github chính thức của Checkstyle có cung cấp sẵn 2 bộ quy tắc kiểm thử phổ biến là [Google Checks](#) và [Sun Checks](#).

Bảng 3.1 — So sánh giữa Google Checks và Sun Checks

Tiêu chí	Google Checks	Sun Checks
Mục tiêu thiết kế	Kế thừa có sửa đổi từ Google Java Style Guide, tập trung và tính nhất quán giữa các dự án lớn và code review dễ dàng, có nhiều quy tắc hiện đại và chi tiết hơn	Tuân thủ code conventions cổ điển của Sun Microsystems (Oracle), nghiêm ngặt hơn trong việc kiểm tra cú pháp và đề cao tính dễ đọc, phù hợp cho các doanh nghiệp truyền thống
Mức độ nghiêm trọng của các vi phạm (Severity)	Warning, phù hợp trong giai đoạn phát triển dần	Error, vi phạm có thể khiến build thất bại
Các quy tắc tổng quát	– Độ dài dòng: 100 ký tự. – Thụt lề: 2 dấu cách (không sử dụng tab). – Bắt buộc có <code>{}</code> kể cả với khối lệnh đơn. – <code>{</code> phải đặt ở cuối dòng khai báo.	– Độ dài dòng: 80 ký tự. – Thụt lề: Sử dụng dấu cách, không có yêu cầu về độ dài. – Có thể bỏ <code>{}</code> với khối lệnh đơn. – <code>{</code> phải đặt ở dòng mới.
Quy ước đặt tên	Dùng nhiều regex phức tạp hơn, chia rõ cho package, class, method, parameter, variable, generic type	Có quy tắc đặt tên tổng quát hơn, ít chi tiết hơn, cho phép mở rộng hơn

Tiêu chí	Google Checks	Sun Checks
JavaDoc	Không bắt buộc JavaDoc cho tất cả method	Yêu cầu JavaDoc cho public method và class
...	...	...

Từ bảng so sánh trên, có thể thấy *Google Checks* linh hoạt, hiện đại và phù hợp với các dự án mã nguồn mở hơn, trong khi *Sun Checks* nghiêm ngặt và phù hợp với các doanh nghiệp truyền thống hơn.

Bên cạnh 2 bộ quy tắc có sẵn, người dùng có thể tự định nghĩa các quy tắc riêng thông qua file XML cấu hình. Cú pháp của file cấu hình XML được mô tả chi tiết trong [tài liệu chính thức của Checkstyle](#).

Một file cấu hình XML bao gồm các **module** (quy tắc kiểm thử) được tổ chức theo cấu trúc cây, trong đó mỗi **module** có thể chứa các **property** (thuộc tính) để tùy chỉnh hành vi của quy tắc đó, và có thể lồng các **module** con bên trong để tạo thành các nhóm quy tắc phức tạp hơn.

Mỗi **module** được phân biệt với nhau bằng trường **name** hoặc **property** có **name** là **id**.

Các **property** thường có 2 trường chính là **name** (tên thuộc tính) và **value** (giá trị thuộc tính).

```
<?xml version="1.0"?>
<!DOCTYPE module PUBLIC
    "-//Checkstyle//DTD Checkstyle Configuration 1.3//EN"
    "https://checkstyle.org/dtds/configuration_1_3.dtd">

<module name="Checker">
  <module name="TreeWalker">
    <property name="tabWidth" value="4"/>
    ...
  </module>
</module>
```

Chương trình 3.1 — Ví dụ về file cấu hình Checkstyle XML

3.2.2. Thực thi kiểm thử

Do dự án MegaBasterd là một dự án mã nguồn mở, được đóng góp bởi nhiều contributor với thói quen code khác nhau, nên nhóm quyết định sử dụng bộ quy tắc *Google Checks* như một tiêu chuẩn để phân tích mã nguồn dự án này.

Sau khi tải file cấu hình `google_checks.xml` và clone mã nguồn dự án MegaBasterd về máy, tiến hành chạy Checkstyle thông qua command line bằng lệnh:

```
java -jar C:\checkstyle\checkstyle-12.1.1-all.jar \
-c C:\checkstyle\google_checks.xml \
-f xml \
-o C:\checkstyle\checkstyle-result.xml \
D:\CODE\Java\megabasterd\src
```

Chương trình 3.2 — Lệnh chạy Checkstyle qua command line

Trong đó:

- `-c` : Chỉ định file cấu hình quy tắc kiểm thử.
- `-f xml` : Chỉ định định dạng đầu ra là XML.
- `-o` : Chỉ định file đầu ra để lưu kết quả kiểm thử.

Bên cạnh những tham số trên, Checkstyle CLI còn hỗ trợ nhiều tham số khác để tùy chỉnh quá trình kiểm thử, được mô tả chi tiết trong [tài liệu chính thức của Checkstyle](#).

Người dùng cũng có thể chạy Checkstyle trên một file cụ thể thay vì toàn bộ thư mục, bằng cách thay thế đường dẫn thư mục `D:\CODE\Java\megabasterd\src` trong lệnh trên bằng đường dẫn file cần kiểm tra, ví dụ:

```
D:\CODE\Java\megabasterd\src\...\AboutDialog.java .
```

3.3. Kết quả kiểm thử

Sau khi chạy, Checkstyle sẽ phân tích toàn bộ file có đuôi `.java`, `.properties` và `.xml` (do cấu hình `fileExtensions` của `google_checks`) trong thư mục `src` của dự án MegaBasterd, và ghi kết quả kiểm thử vào file

`checkstyle-result.xml` . Dưới đây là một phần của file kết quả sau khi thực thi kiểm thử:

```
<?xml version="1.0" encoding="UTF-8"?>
<checkstyle version="12.1.1">
<file name="D:\CODE\Java\megabasterd\src\...\AboutDialog.java">
<error line="10" column="1" severity="warning" message="'package' should be separated
from previous line."
source="com.puppycrawl.tools.checkstyle.checks.whitespace.EmptyLineSeparatorCheck"/>
<error line="12" column="51" severity="warning" message="Using the '.*' form of import
should be avoided - com.tonikelope.megabasterd.MainPanel.*."
source="com.puppycrawl.tools.checkstyle.checks.imports.AvoidStarImportCheck"/>
...
</file>
<file name="D:\CODE\Java\megabasterd\src\...\ChunkDownloader.java">
...
<error line="28" severity="warning" message="Summary javadoc is missing."
source="com.puppycrawl.tools.checkstyle.checks.javadoc.SummaryJavadocCheck"/>
...
<error line="235" column="5" severity="warning" message="'method def rcurl' has
incorrect indentation level 4, expected level should be 2."
source="com.puppycrawl.tools.checkstyle.checks.indentation.IndentationCheck"/>
</file>
<file name="D:\CODE\Java\megabasterd\src\...\MegaProxyServer.java">
...
<error line="72" column="37" severity="warning" message="Empty catch block."
source="com.puppycrawl.tools.checkstyle.checks.blocks.EmptyCatchBlockCheck"/>
...
<error line="93" severity="warning" message="Line is longer than 100 characters (found
129)." source="com.puppycrawl.tools.checkstyle.checks.sizes.LineLengthCheck"/>
...
</file>
<file name="D:\CODE\Java\megabasterd\src\...\UploadView.java">
<error line="16" column="1" severity="warning" message="Import statement for
'java.lang.Integer.MAX_VALUE' is in the wrong order. Should be in the 'STATIC' group,
expecting not assigned imports on this line."
source="com.puppycrawl.tools.checkstyle.checks.imports.CustomImportOrderCheck"/>
<error line="428" column="5" severity="warning" message="WhitespaceAround: '}' is not
followed by whitespace. Empty blocks may only be represented as {} when not part of a
multi-block statement (4.1.3)"
source="com.puppycrawl.tools.checkstyle.checks.whitespace.WhitespaceAroundCheck"/>
<error line="552" column="17" severity="warning" message="Abbreviation in name
'updateCBC' must contain no more than '1' consecutive capital letters."
source="com.puppycrawl.tools.checkstyle.checks.naming.AbbreviationAsWordInNameCheck"/>
...
</file>
</checkstyle>
```

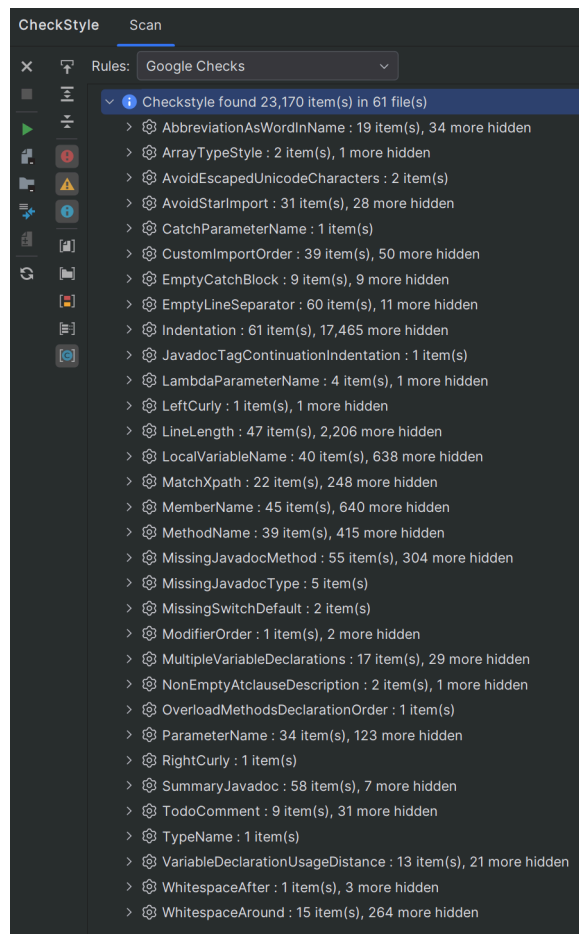
Chương trình 3.3 — Kết quả phân tích mã nguồn dự án MegaBasterd

3.4. Phân tích kết quả

Sau khi phân tích file `checkstyle-result.xml`, ta thấy rằng hầu hết các lỗi vi phạm thuộc dạng *Indentation* (thụt lề) vì dự án MegaBasterd sử dụng 4 space cho mỗi cấp thụt lề, trong khi *Google Checks* yêu cầu 2 space. Ngoài ra còn có một số lỗi khác như:

- *LineLength*: Độ dài dòng vượt quá 100 ký tự.
- *WhitespaceAround*: Thiếu khoảng trắng xung quanh các toán tử.
- *AvoidStarImport*: Sử dụng `import` dạng `.*`.
- *EmptyCatchBlock*: Khối `catch` trống.
- *SummaryJavadoc*: Thiếu JavaDoc tóm tắt cho class hoặc method
- Các lỗi tên biến, tên phương thức không tuân thủ quy ước đặt tên của Google.

Để tổng quát hơn, ta phân tích kèm với Plugin Checkstyle-IDEA trên IntelliJ IDEA:



Hình 3.1 — Tổng quan các lỗi vi phạm trong dự án MegaBasterd

Có thể thấy, trong dự án MegaBasterd có tổng cộng 23170 vi phạm trong 61/61 file mã nguồn. Trong đó, lỗi phổ biến nhất vẫn là *Indentation* (17526 vi phạm trong 61/61 file), theo sau là *Linelenhth* (2253 vi phạm trong 47/61 file) và *MemberName* (685 vi phạm trong 35/61 file),...

Điều này cho thấy dự án MegaBasterd không sử dụng Code Convention của Google, dẫn đến việc vi phạm nhiều quy tắc kiểm thử của Google Checks.

3.5. Đề xuất khắc phục

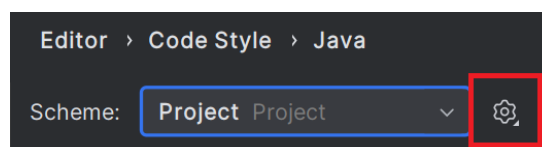
Để giảm thiểu các lỗi vi phạm được phát hiện bởi Checkstyle, có thể chỉnh sửa file cấu hình `google_checks.xml` để bỏ qua một số quy tắc không phù hợp với dự án, ví dụ như *Indentation* hoặc *LineLength* và giữ lại những quy tắc quan trọng. Hoặc ta có thể format lại mã nguồn dự án để tuân thủ các quy tắc của Google Checks.

Có 2 cách để format lại mã nguồn dự án:

- Sử dụng tính năng Reformat Code có sẵn trong IntelliJ IDEA kèm với file cấu hình [Google Java Style](#).
- Sử dụng plugin [google-java-format](#) để tự động format mã nguồn theo chuẩn Google.

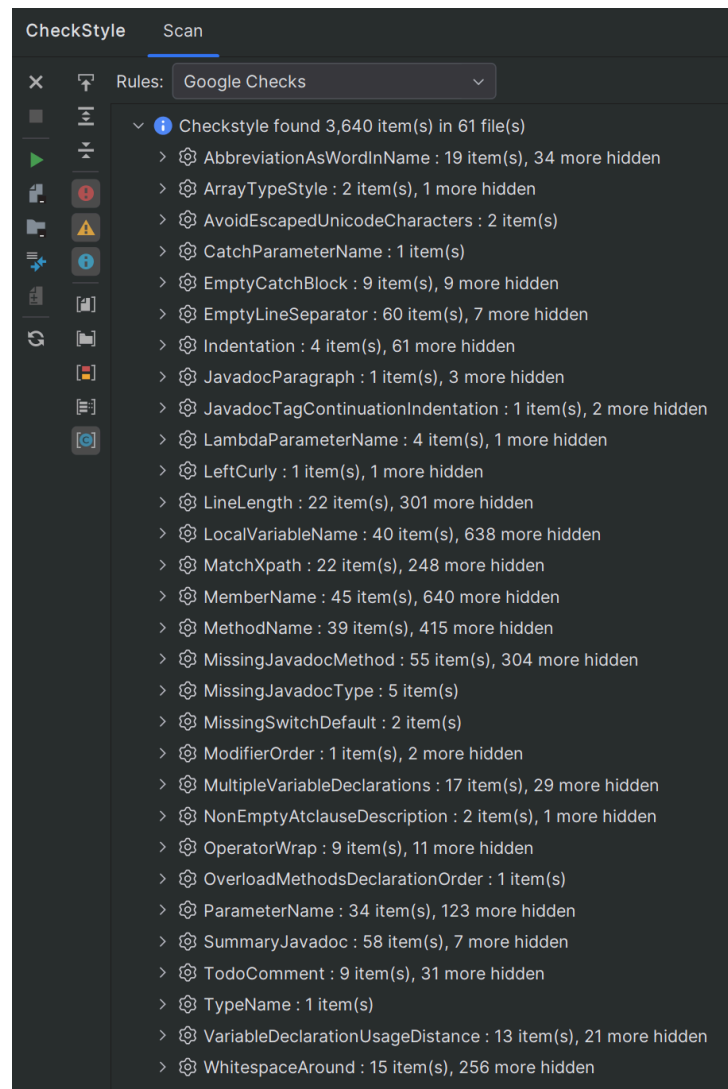
Nhóm sẽ thử áp dụng cách thứ nhất để format lại mã nguồn dự án MegaBasterd và chạy lại Checkstyle để kiểm tra kết quả.

1. Tải file cấu hình [intellij-java-google-style.xml](#).
2. Trong IntelliJ IDEA, vào **File** → **Settings** → **Editor** → **Code Style** → **Java**.
3. Nhấn vào biểu tượng bánh răng và chọn **Import Scheme** → **IntelliJ IDEA code style XML**.



4. Chọn file cấu hình đã tải về và nhấn **OK**.
5. Mở dự án MegaBasterd và sử dụng tính năng Reformat Code để tự động định dạng lại mã nguồn theo chuẩn Google.

Sau khi format lại mã nguồn và chạy lại Checkstyle, thu được [file kết quả](#). Sử dụng Plugin Checkstyle-IDEA để tổng quan hóa kết quả:



Có thể thấy số lượng vi phạm đã giảm đáng kể từ 23170 xuống còn 3640 vi phạm, trong đó lỗi *Indentation* đã không còn xuất hiện nữa. Thay vào đó là các lỗi như *MemberName* (685 vi phạm), *LocalVariableName* (678 vi phạm), *MethodName* (454 vi phạm),...

Kiểm tra nhanh file `APIException.java` vi phạm lỗi *MemberName*:

```
...  
public abstract class APIException extends Exception {  
  
    protected Integer _code;  
  
    ...  
}
```

Tên biến `_code` không tuân thủ quy ước đặt tên của Google (sử dụng dấu gạch dưới ở đầu tên biến), có thể ý đồ của tác giả là để biểu thị đây là biến protected. Để khắc phục lỗi này, có thể đổi tên biến thành `code`.

Chương 4

Kết luận

4.1. Đánh giá hiệu quả

4.2. Kết luận

